

Reconfigurable Computing — Post Mid-Semester Lecture Notes (Lectures 19–26)

Based on course slides by Dr. Meetha V. Shenoy

Contents

1	Lecture 19: FPGA Routing and Architecture	2
1.1	Overview of FPGA CAD Flow	2
1.2	Island-Style FPGA Architecture	2
1.3	Connection Block Flexibility	2
1.4	Switch Box Types	2
1.5	Routing as a Grid Graph	3
1.6	Maze Routing Algorithm	3
1.7	Global vs Detailed Routing	3
1.8	Pathfinder Algorithm	3
2	Lecture 20: ZedBoard, Zynq SoC and JTAG	4
2.1	ZedBoard Overview	4
2.2	Zynq-7000 Architecture	4
2.3	JTAG and IEEE 1149.1	4
2.4	JTAG Instructions	4
2.5	Programming Methods	4
3	Lecture 21: Security of FPGA-Based Systems (Part 1)	5
3.1	Supply Chain Model	5
3.2	Threat: Cloning	5
3.3	Identification: Friend or Foe	5
3.4	Physically Unclonable Functions (PUFs)	5
3.5	Hardware Trojans	5
3.6	Backdoor Communication	5
4	Lecture 22: Security of FPGA-Based Systems (Part 2)	6
4.1	Replay Attacks	6
4.2	Secure Boot	6
4.3	Reverse Engineering of ASICs	6
4.4	IC Camouflage	6
4.5	Logic Obfuscation	6
4.6	Tamper Detection	6

5	Lecture 23: Coarse-Grained Reconfigurable Architectures (CGRAs)	7
5.1	CGRA Structure	7
5.2	Processing Elements	7
5.3	Execution Controller	7
5.4	CGRAs vs FPGAs	7
5.5	Interconnect Types	7
6	Lecture 24–26: Configurable Logic Block (CLB) Architecture	8
6.1	CLB Structure	8
6.2	LUT Functionality	8
6.3	LUTs as Multiplexers	8
6.4	Storage Elements	8
6.5	Distributed RAM	8
6.6	Shift Register LUTs (SRLs)	8
6.7	Block RAM vs Distributed RAM	9
6.8	Pipelining	9
7	Lecture 26 Addendum: FPGA Implementation of 3×3 Convolution	10
7.1	Problem Definition	10
7.2	Resource Usage	10
7.3	Pipeline Stages	10

1 Lecture 19: FPGA Routing and Architecture

1.1 Overview of FPGA CAD Flow

According to the flow diagram shown in the slides (Design Entry → Synthesis → Logic Optimization → Mapping to k-LUT → Packing LUTs to CLBs → Placement → Routing → Simulation → Configuration) :contentReference[oaicite:1]index=1, the FPGA compilation process proceeds through the following steps:

1. **Design Entry:** HDL / schematic input.
2. **Synthesis:** Converts HDL to gate-level netlist.
3. **Logic Optimization.**
4. **Technology Mapping:** Map to k-input LUTs.
5. **Packing:** Pack LUTs into CLBs.
6. **Placement:** Assign CLBs to FPGA grid.
7. **Routing:** Connect all placed blocks using routing fabric.
8. **Bitstream Generation.**

1.2 Island-Style FPGA Architecture

The slides depict an island-style FPGA (logic blocks surrounded by routing channels and switch boxes) :contentReference[oaicite:2]index=2.

- **Logic Blocks (L)** implement combinational/sequential logic.
- **Connection Boxes (C)** connect CLB pins to routing tracks.
- **Switch Boxes (S)** connect routing tracks to each other.

1.3 Connection Block Flexibility

The connection block has a parameter:

$$F_c = \text{number of routing tracks each CLB pin may connect to}$$

Higher F_c increases routability but increases area. :contentReference[oaicite:3]index=3

1.4 Switch Box Types

Switch boxes connect horizontal and vertical tracks. Two major types shown in the slides are :contentReference[oaicite:4]index=4:

- **Planar / Subset switch box:** Only maps track i to track i (0–0, 1–1).
- **Wilton switch box:** Permits permutation across domains (0–3, 1–2, etc.).

1.5 Routing as a Grid Graph

Slides describe routing as a graph where:

- Nodes = switches, pins, wire segments.
- Obstacles = used routing resources.
- Cost = wirelength + congestion + timing. :contentReference[oaicite:5]index=5

1.6 Maze Routing Algorithm

A BFS/Dijkstra style expansion is used:

1. Initialize grid.
2. Expand wave from source.
3. When destination reached, backtrack.
4. Optimize for congestion/timing. :contentReference[oaicite:6]index=6

1.7 Global vs Detailed Routing

- **Global routing:** Choose coarse path (which channels).
- **Detailed routing:** Assign specific tracks and switches. :contentReference[oaicite:7]index=7

1.8 Pathfinder Algorithm

Iterative negotiated routing:

1. Initial shortest-path routing.
2. Compute congestion.
3. Increase cost of overused wires.
4. Rip-up and reroute.
5. Iterate until convergence. :contentReference[oaicite:8]index=8

2 Lecture 20: ZedBoard, Zynq SoC and JTAG

2.1 ZedBoard Overview

Slides provide a labeled diagram of the ZedBoard main components (USB-JTAG, USB-UART, GPIO, HDMI, Ethernet, Memory interfaces) :contentReference[oaicite:9]index=9.

2.2 Zynq-7000 Architecture

Zynq integrates:

- **Processing System (PS)**: Dual-core ARM Cortex-A9.
- **Programmable Logic (PL)**: Artix-7 FPGA fabric.
- **AXI Interconnect**: Communication between PS and PL. :contentReference[oaicite:10]index=10

2.3 JTAG and IEEE 1149.1

The wrapper architecture figure shows boundary scan cells around the internal logic :contentReference[oaicite:11]index=11.

Mandatory pins

- TCK, TMS, TDI, TDO, optional TRST. :contentReference[oaicite:12]index=12

2.4 JTAG Instructions

- **EXTEST**: Tests external connections (figure in slides).
- **BYPASS**: 1-bit bypass register.
- **INTTEST**: Test internal logic. :contentReference[oaicite:13]index=13

2.5 Programming Methods

ZedBoard supports programming via:

1. USB-JTAG (default).
2. JTAG header.
3. SPI Flash.
4. SD Card. :contentReference[oaicite:14]index=14

3 Lecture 21: Security of FPGA-Based Systems (Part 1)

3.1 Supply Chain Model

Slides show interactions between foundries, FPGA vendors, system developers, IP vendors, end users :contentReference[oaicite:15]index=15.

3.2 Threat: Cloning

FPGAs load bitstreams from external flash; attacker can read this bitstream and reprogram unauthorized devices :contentReference[oaicite:16]index=16.

3.3 Identification: Friend or Foe

Authentication protocol:

1. FPGA sends challenge C to EEPROM.
2. EEPROM computes $\text{Hash}(K_{\text{EEPROM}}, C)$.
3. FPGA computes $\text{Hash}(K_{\text{FPGA}}, C)$.
4. If match $\Rightarrow$ valid device. :contentReference[oaicite:17]index=17

3.4 Physically Unclonable Functions (PUFs)

Key points from the slides:

- Ring oscillators differ due to manufacturing variations.
- Frequency differences yield unique response bits.
- Multiple challenges produce a device fingerprint. :contentReference[oaicite:18]index=18

3.5 Hardware Trojans

A malicious modification consisting of:

- **Trigger:** Rare conditions.
- **Payload:** Key leakage, disabling chip, reduced performance. :contentReference[oaicite:19]index=19

3.6 Backdoor Communication

Unauthorized hidden channels inside FPGA IP blocks. :contentReference[oaicite:20]index=20

4 Lecture 22: Security of FPGA-Based Systems (Part 2)

4.1 Replay Attacks

Attacker replays an older bitstream version to exploit vulnerabilities :contentReference[oaicite:21]index=21.
PUF-based license generation:

$$\text{License}_1 = f(R, \text{Version}, \text{Secret})$$

Bitstream executes only when PUF-derived license matches. :contentReference[oaicite:22]index=22

4.2 Secure Boot

Slides illustrate standard secure-boot flow:

- Host signs image.
- Device verifies signature using public key in fuses. :contentReference[oaicite:23]index=23

4.3 Reverse Engineering of ASICs

Uses delayering, SEM imaging, and X-ray tomography to restore netlists. :contentReference[oaicite:24]index=24

4.4 IC Camouflage

Gates that appear identical physically but behave differently. :contentReference[oaicite:25]index=25

4.5 Logic Obfuscation

Insert key-dependent gates; wrong keys produce incorrect behavior. :contentReference[oaicite:26]index=26

4.6 Tamper Detection

Secure FPGAs include:

- Light sensors.
- Switches.
- Vibration detectors.
- Zeroization of secrets when triggered. :contentReference[oaicite:27]index=27

5 Lecture 23: Coarse-Grained Reconfigurable Architectures (CGRAs)

5.1 CGRA Structure

Slides depict Processing Element (PE) arrays, register files, configuration memories, host processor
:contentReference[oaicite:28]index=28.

5.2 Processing Elements

Each PE contains:

- ALU + shifter.
- Local register file.
- Shared register for neighbor communication. :contentReference[oaicite:29]index=29

5.3 Execution Controller

Controls:

- PE activation schedule.
- Context switching.
- Pipeline start/stop.
- Synchronization. :contentReference[oaicite:30]index=30

5.4 CGRAs vs FPGAs

- FPGA: logic-level programming.
- CGRA: operation-level programming (dataflow-like). :contentReference[oaicite:31]index=31

5.5 Interconnect Types

1. **Nearest Neighbor.**
2. **Next-Hop / Flexible Switchbox.**
3. **Bus-Based Interconnect.**
4. **Augmented Interconnect.** :contentReference[oaicite:32]index=32

6 Lecture 24–26: Configurable Logic Block (CLB) Architecture

6.1 CLB Structure

Slides show:

- 1 CLB = 2 slices.
- Each slice = 4 LUTs + 8 FFs + carry chain + multiplexers. :contentReference[oaicite:33]index=33

There are two slice types:

- **SLICEM**: supports distributed RAM + SRL.
- **SLICEL**: logic only. :contentReference[oaicite:34]index=34

6.2 LUT Functionality

- 6-input LUT with output O_6 .
- Can act as two 5-input LUTs using O_5 . :contentReference[oaicite:35]index=35

6.3 LUTs as Multiplexers

Slides illustrate 4:1, 8:1, and 16:1 multiplexer construction using LUTs :contentReference[oaicite:36]index=36.

6.4 Storage Elements

- Each LUT output feeds optional FF.
- Total: 8 FFs per slice. :contentReference[oaicite:37]index=37

6.5 Distributed RAM

Available only in SLICEM. Good for:

- Small tables.
- Coefficients.
- Low-latency lookup. :contentReference[oaicite:38]index=38

6.6 Shift Register LUTs (SRLs)

SRL32

A LUT can become a 32-bit shift register:

- Depth = 1–32.
- Address lines A[4:0] select tap.
- Q31 provides cascading. :contentReference[oaicite:39]index=39

SRL Cascading

Up to 128 cycles using 4 LUTs (SRL1–SRL4). Tables and diagrams shown in slides illustrate ranges
:contentReference[oaicite:40]index=40.

6.7 Block RAM vs Distributed RAM

Characteristic	Distributed RAM	Block RAM
Depth	≤ 128	large (512+)
Width	≤ 8 bits	up to 72 bits
Read	async	sync
Location	LUT fabric	BRAM tile

:contentReference[oaicite:41]index=41

6.8 Pipelining

Breaking long combinational paths into stages improves:

- Frequency.
- Throughput.
- Timing closure. :contentReference[oaicite:42]index=42

Example:

$$r = (a + b) \cdot c \quad \Rightarrow \quad \begin{array}{l} \text{Stage 1: } a + b \\ \text{Stage 2: } (a + b) \cdot c \end{array}$$

7 Lecture 26 Addendum: FPGA Implementation of 3×3 Convolution

7.1 Problem Definition

Compute 3×3 convolution over 28×28 image:

- 9 multiplies + 8 adds/pixel. :contentReference[oaicite:43]index=43

7.2 Resource Usage

- **Line buffers:** BRAM.
- **Kernel coefficients:** Distributed RAM / registers.
- **Window shift:** SRL32.
- **Multiply:** DSP48E1.
- **Adder tree:** CARRY4 chain. :contentReference[oaicite:44]index=44

7.3 Pipeline Stages

1. Window extraction (BRAM/RAM).
2. 9 parallel multiplications.
3. Row partial sums.
4. Final accumulation.
5. Activation/normalization.
6. Output store. :contentReference[oaicite:45]index=45